.

ESTRUTURA DE DADOS
ESTUDO DIRIGIDO
Recursão em Java

ORIENTAÇÕES PARA O ESTUDO AUTÔNOMO

Este material foi estruturado para que você aprenda recursão de forma autônoma, através de leitura orientada, experimentação prática e reflexão crítica. O estudo está organizado em **trilhas de aprendizagem** que você deve percorrer sequencialmente.

⏱ **Tempo estimado total: 3,5 horas**

Como usar este material

- ➔ Consulte a apostila sobre recursão disponível junto com este estudo dirigido
- ➔ Use as Ferramentas de Visualização e Simulação indicadas no final da apostila sobre recursão
- ➔ Leia cada seção completamente antes de passar para as atividades práticas.
- ➔ Execute todos os códigos de exemplo no seu ambiente de desenvolvimento.
- ➔ Complete as atividades práticas marcadas com ✏️ antes de avançar.
- ➔ Use o checklist de progresso ao final para verificar sua compreensão.

TRILHA 1 – FUNDAMENTOS DA RECURSÃO (30 min)

🎯 **Objetivo:** Compreender o conceito de recursão, identificar seus componentes essenciais e executar manualmente o primeiro exemplo recursivo.

1.1 O Que é Recursão?

Leia atentamente:

Recursão é uma técnica onde uma função resolve um problema **chamando a si mesma** para resolver versões menores do mesmo problema. É análoga à definição matemática por indução:

Exemplo matemático: Fatorial

```
n! = n × (n-1)!    (definição recursiva)
0! = 1             (caso base)
```

Três componentes obrigatórios de toda função recursiva:

Componente	Descrição
1. Caso Base	Condição de parada que resolve o problema diretamente, SEM recursão. Exemplo: fatorial(0) = 1
2. Caso Recursivo	Chamada da função a si mesma com entrada modificada. Exemplo: fatorial(n) chama fatorial(n-1)
3. Progresso	Cada chamada recursiva se aproxima do caso base. Exemplo: n → n-1 → n-2 → ... → 0

1.2 Primeiro Exemplo: Fatorial

📖 **Leia e digite este código no seu IDE:**

```
public class ExemploRecursao {

    public static int fatorial(int n) {
        // Validação
        if (n < 0) {
            throw new IllegalArgumentException("n >= 0");
        }
        // CASO BASE: fatorial de 0 ou 1 é 1
```

```
        if (n == 0 || n == 1) {
            System.out.println("Caso base: " + n + "! = 1");
            return 1;
        }
        // CASO RECURSIVO: n! = n × (n-1)!
        System.out.println("Calculando " + n + "!");
        int resultado = n * fatorial(n - 1);
        System.out.println(n + "! = " + resultado);
        return resultado;
    }

    public static void main(String[] args) {
        System.out.println("Resultado final: " + fatorial(4));
    }
}
```

✏️ ATIVIDADE PRÁTICA 1.1 – Execute e Observe

1. Digite o código acima no seu IDE
2. Execute o programa e observe a saída no console
3. Identifique na saída: onde está o caso base? Quantas chamadas recursivas foram feitas?
4. Experimente: mude o main para calcular fatorial(5). O que muda?
5. Teste o caso limite: fatorial(0). Funciona corretamente?

1.3 Execução Manual (Traço de Execução)

Para compreender profundamente recursão, você precisa **simular mentalmente** como a JVM executa o código. Vamos fazer isso passo a passo para fatorial(3):

```
CHAMADA INICIAL: fatorial(3)
│
├─ n = 3, não é caso base
├─ Preciso calcular: 3 * fatorial(2)
│  │
│  └─ CHAMADA RECURSIVA: fatorial(2)
│     │
│     ├─ n = 2, não é caso base
│     ├─ Preciso calcular: 2 * fatorial(1)
```

```
│     │  │
│     │  └─ CHAMADA RECURSIVA: fatorial(1)
│     │     │
│     │     ├─ n = 1, É CASO BASE!
│     │     └─ RETORNA: 1
│     │
│     ├─ Agora posso calcular: 2 * 1 = 2
│     └─ RETORNA: 2
│
├─ Agora posso calcular: 3 * 2 = 6
└─ RETORNA: 6
```

✏️ ATIVIDADE PRÁTICA 1.2 – Traço Manual

Pegue papel e caneta. Faça o traço de execução completo (como acima) para fatorial(5). Identifique:

- Quantas chamadas recursivas acontecem?
- Qual é a primeira chamada que retorna um valor?
- Em que ordem os valores vão sendo calculados?

Tempo estimado: 10 minutos

✅ Checkpoint da Trilha 1:

- ☐ Sei identificar caso base, caso recursivo e progresso em uma função
- ☐ Consigo executar manualmente um código recursivo simples
- ☐ Entendo que recursão tem fase de expansão (chamadas) e contração (retornos)

TRILHA 2 – PILHA DE CHAMADAS E STACKOVERFLOW (25 min)

🎯 **Objetivo:** Compreender como a JVM gerencia chamadas recursivas através da pilha e aprender a prevenir erros de estouro.

2.1 A Pilha de Chamadas (Call Stack)

📖 **Conceito fundamental:**

A JVM mantém uma estrutura de pilha (LIFO - Last In, First Out) onde cada chamada de método adiciona um stack frame. Cada frame armazena:

- Parâmetros do método (ex: n = 3)
- Variáveis locais (ex: resultado)
- Endereço de retorno (onde continuar após o método terminar)

Visualização da pilha durante fatorial(3):

```
┌──────────────────────────┐
│ fatorial(1) n=1          │ ← TOPO (caso base)
├──────────────────────────┤   Retorna 1
│ fatorial(2) n=2          │
├──────────────────────────┤   Aguardando fatorial(1)
│ fatorial(3) n=3          │
├──────────────────────────┤   Aguardando fatorial(2)
│ main()                   │
└──────────────────────────┘ ← BASE
```

2.2 StackOverflowError – O Que é e Como Prevenir

A pilha tem tamanho limitado (geralmente 512KB a 1MB). Se a recursão for muito profunda, a pilha estoura, causando `StackOverflowError`.

⚠️ **EXEMPLO DE CÓDIGO INCORRETO (NÃO EXECUTE!):**

```
// ERRO: Falta caso base
public int fatorialErrado(int n) {
    return n * fatorialErrado(n - 1);  // Recursão infinita!
}
```

Causas comuns:

- **Falta de caso base** (como acima)
- **Caso base inalcançável** (condição que nunca é satisfeita)

- **Profundidade excessiva** (ex: Fibonacci ingênuo para n > 40)

✏️ **ATIVIDADE PRÁTICA 2.1 – Corrija o Erro**

O código abaixo tem um bug que causa StackOverflowError. Identifique e corrija:

```
public int somaAteN(int n) {
    if (n == 1) return 1;
    return n + somaAteN(n - 1);
}// Problema: somaAteN(0) causa erro!
```

Dica: O caso base atual só funciona se n >= 1. Como tratar n = 0 ou n < 0?

✅ **Checkpoint da Trilha 2:**

- ☐ Compreendo que cada chamada recursiva adiciona um frame à pilha
- ☐ Sei identificar código que causaria StackOverflowError
- ☐ Consigo escrever casos base robustos que previnem erros

TRILHA 3 – RELAÇÕES DE RECORRÊNCIA E TEOREMA MESTRE (45 min)

🎯 **Objetivo:** Aprender a analisar a complexidade de algoritmos recursivos através de relações de recorrência e aplicar o Teorema Mestre para determinação de complexidade assintótica.

3.1 O Que São Relações de Recorrência?

📖 **Conceito fundamental:**

Uma **relação de recorrência** expressa o tempo de execução de um algoritmo recursivo em termos de seu tempo de execução em entradas menores. É a ferramenta matemática para análise de complexidade de recursão.

Forma geral:

T(n) = tempo para resolver um problema de tamanho n
T(n) = [trabalho fora da recursão] + [trabalho das chamadas recursivas]

3.2 Exemplo 1: Fatorial

Código:

```
public int fatorial(int n) {
    if (n <= 1) return 1;          // O(1)
    return n * fatorial(n-1);       // O(1) + T(n-1)
}
```

Relação de recorrência:

```
T(n) = T(n-1) + O(1)
T(1) = O(1)  (caso base)
```

Expansão manual:

```
T(n) = T(n-1) + c
     = T(n-2) + c + c
     = T(n-3) + c + c + c
     = ...
     = T(1) + c·(n-1)
     = O(1) + O(n)
     = O(n)
```

Conclusão: Fatorial recursivo tem complexidade **O(n)** temporal e **O(n)** espacial (pela pilha).

3.3 Exemplo 2: Fibonacci Ingênuo

Código:

```
public int fibonacci(int n) {
    if (n <= 1) return n;                          // O(1)
    return fibonacci(n-1) + fibonacci(n-2);        // T(n-1) + T(n-2)
}
```

Relação de recorrência: Esta recorrência é mais complexa. Usando análise por árvore de recursão, podemos demonstrar que:

```
T(n) ≈ 2^n
```

Conclusão: Fibonacci ingênuo tem complexidade **O(2^n)** — exponencial e inviável para n > 40.

3.4 Exemplo 3: Busca Binária Recursiva

Código:

```
public int buscaBinaria(int[] arr, int alvo, int esq, int dir) {
    if (esq > dir) return -1;                      // O(1)

    int meio = esq + (dir - esq) / 2;              // O(1)

    if (arr[meio] == alvo) return meio;            // O(1)

    if (arr[meio] < alvo) {
        return buscaBinaria(arr, alvo, meio+1, dir);  // T(n/2)
    } else {
        return buscaBinaria(arr, alvo, esq, meio-1);  // T(n/2)
    }
}
```

Relação de recorrência:

```
T(n) = T(n/2) + O(1)
```

```
T(1) = O(1)
```

Expansão:

```
T(n) = T(n/2) + c
     = T(n/4) + c + c
     = T(n/8) + c + c + c
     = ...
     = T(1) + c·log₂(n)
     = O(log n)
```

Conclusão: Busca binária tem complexidade **O(log n)**.

3.5 O Teorema Mestre

O **Teorema Mestre** é uma ferramenta que permite determinar a complexidade de recorrências do tipo divide-and-conquer sem precisar fazer expansão manual.

Forma canônica:

$T(n) = a \cdot T(n/b) + f(n)$

onde:
$a \geq 1$ → número de subproblemas recursivos
$b > 1$ → fator de divisão do problema
$f(n)$ → custo de dividir e combinar (trabalho fora da recursão)

Os três casos do Teorema Mestre:

Seja **c = log_b(a)** (expoente crítico):

Caso 1: Se $f(n) = O(n^d)$ onde $d < c$, então:

$T(n) = \Theta(n^c) = \Theta(n^{(\log_b(a))})$

Interpretação: O trabalho recursivo domina.

Caso 2: Se $f(n) = \Theta(n^c \cdot \log^k(n))$ onde $k \geq 0$, então:

$T(n) = \Theta(n^c \cdot \log^{(k+1)}(n))$

Interpretação: Trabalho recursivo e não-recursivo são balanceados.

Caso 3: Se $f(n) = \Omega(n^d)$ onde $d > c$, e se $a \cdot f(n/b) \leq k \cdot f(n)$ para alguma constante $k < 1$ e n suficientemente grande, então:

$T(n) = \Theta(f(n))$

Interpretação: O trabalho não-recursivo domina.

3.6 Aplicando o Teorema Mestre: Exemplos

Exemplo 1: Busca Binária

$T(n) = T(n/2) + O(1)$

$a = 1$, $b = 2$, $f(n) = O(1) = O(n^0)$
$c = \log_2(1) = 0$

$f(n) = O(n^0) = O(n^c) \rightarrow$ CASO 2 (com $k=0$)
$T(n) = \Theta(n^0 \cdot \log^1(n)) = \Theta(\log n)$

Exemplo 2: Merge Sort

$T(n) = 2 \cdot T(n/2) + O(n)$

$a = 2$, $b = 2$, $f(n) = O(n) = O(n^1)$
$c = \log_2(2) = 1$

$f(n) = O(n^1) = O(n^c) \rightarrow$ CASO 2 (com $k=0$)
$T(n) = \Theta(n^1 \cdot \log^1(n)) = \Theta(n \log n)$

Exemplo 3: Multiplicação de Karatsuba

$T(n) = 3 \cdot T(n/2) + O(n)$

$a = 3$, $b = 2$, $f(n) = O(n)$
$c = \log_2(3) \approx 1.585$

$f(n) = O(n^1)$, mas $d=1 < c=1.585 \rightarrow$ CASO 1
$T(n) = \Theta(n^{\log_2(3)}) \approx \Theta(n^{1.585})$

Exemplo 4: Algoritmo Fictício

$T(n) = 2 \cdot T(n/2) + O(n^2)$

$a = 2$, $b = 2$, $f(n) = O(n^2)$
$c = \log_2(2) = 1$

$f(n) = O(n^2)$, mas $d=2 > c=1$ → CASO 3
Verificar condição de regularidade: $2 \cdot f(n/2) = 2 \cdot (n/2)^2 = n^2/2 \leq k \cdot n^2$ para $k=1/2 < 1$ ✓
$T(n) = \Theta(n^2)$

> ✏️ **ATIVIDADE PRÁTICA 3.1 – Aplique o Teorema Mestre**
>
> Para cada recorrência abaixo, identifique a, b, f(n), calcule c = log_b(a), determine qual caso do Teorema Mestre se aplica e conclua a complexidade T(n):
>
> 1. $T(n) = 4 \cdot T(n/2) + O(n)$
> 2. $T(n) = T(n/3) + O(1)$
> 3. $T(n) = 3 \cdot T(n/4) + O(n \log n)$
> 4. $T(n) = 2 \cdot T(n/2) + O(n \log n)$

3.7 Quando o Teorema Mestre NÃO se aplica

O Teorema Mestre só funciona para recorrências na forma $T(n) = a \cdot T(n/b) + f(n)$. Ele **NÃO se aplica** a:

Fibonacci: $T(n) = T(n-1) + T(n-2) + O(1)$
(Duas subtrações diferentes, não divisão)

Fatorial: $T(n) = T(n-1) + O(1)$
(Subtração linear, não divisão)

Quicksort (pior caso): $T(n) = T(n-1) + T(0) + O(n)$
(Partição desbalanceada)

Para estes casos, use **expansão manual** ou **método da árvore de recursão**.

✅ Checkpoint da Trilha 3:

- ☐ Sei escrever a relação de recorrência de um algoritmo recursivo
- ☐ Compreendo o Teorema Mestre e seus três casos
- ☐ Consigo aplicar o Teorema Mestre para determinar complexidade

TRILHA 4 – PADRÕES RECURSIVOS E OTIMIZAÇÃO (35 min)

🎯 **Objetivo:** Aprender padrões recursivos comuns e técnicas de otimização (memoization).

4.1 Fibonacci: O Problema Clássico

A sequência de Fibonacci é definida como:

```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n-1) + fib(n-2)  para n ≥ 2
```

Implementação ingênua (INEFICIENTE):

```
public static int fibonacciIngenuo(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacciIngenuo(n-1) + fibonacciIngenuo(n-2);
}
// Complexidade: O(2^n) - EXPONENCIAL!
```

Por que é ineficiente?
Porque recalcula os mesmos valores múltiplas vezes. Para fib(5):

```
fib(5)
├─ fib(4)
│  ├─ fib(3)
│  │  └─ fib(2)   ← calculado
│  │      └─ fib(1)
│  └─ fib(2)      ← RECALCULADO!
└─ fib(3)         ← RECALCULADO!
   ├─ fib(2)      ← RECALCULADO!
        └─ fib(1)
```

4.2 Solução: Memoization

Memoization (programação dinâmica) armazena resultados já calculados para evitar recomputação:

```java
import java.util.HashMap;
import java.util.Map;

public class FibonacciOtimizado {
    private static Map<Integer, Long> memo = new HashMap<>();

    public static long fibonacci(int n) {
        // Casos base
        if (n == 0) return 0;
        if (n == 1) return 1;

        // Verifica se já foi calculado
        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        // Calcula e armazena
        long resultado = fibonacci(n-1) + fibonacci(n-2);
        memo.put(n, resultado);

        return resultado;
    }
}
// Complexidade: O(n) tempo, O(n) espaço
```

✏️ ATIVIDADE PRÁTICA 4.1 – Compare Performance

1. Implemente AMBAS as versões: ingênua e com memoization

2. Meça o tempo para calcular fib(35) e fib(40) em cada versão:

```java
long inicio = System.nanoTime();
long resultado = fibonacci(40);
long fim = System.nanoTime();
System.out.println("Tempo: " + (fim-inicio)/1_000_000 + "ms");
```

3. Compare: qual é a diferença de tempo? Por que memoization é muito mais rápida?

✅ **Checkpoint da Trilha 4:**

- ☐ Compreendo por que Fibonacci ingênuo é $O(2^n)$
- ☐ Sei implementar memoization usando HashMap
- ☐ Entendo como memoization reduz complexidade para O(n)

TRILHA 5 – TORRE DE HANÓI (40 min)

🎯 **Objetivo:** Compreender o problema clássico da Torre de Hanói, sua solução recursiva elegante e analisar sua complexidade exponencial.

5.1 O Problema da Torre de Hanói

📖 **Contexto histórico e enunciado:**

A Torre de Hanói é um problema matemático criado pelo matemático francês Édouard Lucas em 1883. O problema consiste em:

Configuração inicial:

- 3 pinos: Origem, Auxiliar e Destino
- n discos de tamanhos diferentes empilhados no pino Origem em ordem decrescente (maior embaixo)

Objetivo:

- Mover todos os discos do pino Origem para o pino Destino

Regras:

1. Mover apenas um disco por vez
2. Um disco só pode ser colocado sobre outro maior (nunca sobre um menor)
3. Pode-se usar o pino Auxiliar como espaço temporário

Visualização para n = 3:

```
Estado inicial:

Origem    Auxiliar    Destino
  |          |          |
 [1]         |          |
 [2]         |          |
 [3]         |          |
-----      -----      -----
```

```
Estado final:

Origem    Auxiliar    Destino
  |          |          |
  |          |         [1]
  |          |         [2]
  |          |         [3]
-----      -----      -----
```

5.2 A Solução Recursiva

A beleza da Torre de Hanói está na elegância de sua solução recursiva. A estratégia se baseia em dividir o problema:

Estratégia para mover n discos da Origem para o Destino:

1. **Mover n-1 discos** da Origem para o Auxiliar (usando Destino como espaço temporário)
2. **Mover o disco maior** (disco n) da Origem para o Destino
3. **Mover n-1 discos** do Auxiliar para o Destino (usando Origem como espaço temporário)

Caso base: Se n = 1, mover diretamente da Origem para o Destino.

Implementação em Java:

```
public class TorreHanoi {


    // Contador de movimentos
    private static int contadorMovimentos = 0;


    /**
     * Move n discos do pino origem para o pino destino usando o pino auxiliar.
     *
     * @param n número de discos a mover
     * @param origem pino de origem
     * @param destino pino de destino
     * @param auxiliar pino auxiliar
     */
    public static void resolverHanoi(int n, char origem, char destino, char
auxiliar) {
        // CASO BASE: se há apenas 1 disco, mover diretamente
        if (n == 1) {
            System.out.println("Mover disco 1 de " + origem + " para " +
destino);
            contadorMovimentos++;
            return;
        }


        // PASSO 1: Mover n-1 discos de origem para auxiliar (usando destino)
        resolverHanoi(n-1, origem, auxiliar, destino);


        // PASSO 2: Mover o disco maior (disco n) de origem para destino
        System.out.println("Mover disco " + n + " de " + origem + " para " +
destino);
        contadorMovimentos++;
```

```
        // PASSO 3: Mover n-1 discos de auxiliar para destino (usando origem)
        resolverHanoi(n-1, auxiliar, destino, origem);
    }

    public static void main(String[] args) {
        int n = 3;  // Número de discos

        System.out.println("Resolvendo Torre de Hanói com " + n + " discos:");
        System.out.println("================================================");

        contadorMovimentos = 0;
        resolverHanoi(n, 'A', 'C', 'B');  // A=Origem, C=Destino, B=Auxiliar

        System.out.println("================================================");
        System.out.println("Total de movimentos: " + contadorMovimentos);
        System.out.println("Fórmula: 2^n - 1 = 2^" + n + " - 1 = " +
((int)Math.pow(2, n) - 1));
    }
}
```

Saída para n = 3:

```
Resolvendo Torre de Hanói com 3 discos:
================================================
Mover disco 1 de A para C
Mover disco 2 de A para B
Mover disco 1 de C para B
Mover disco 3 de A para C
Mover disco 1 de B para A
Mover disco 2 de B para C
Mover disco 1 de A para C
================================================
Total de movimentos: 7
Fórmula: 2^n - 1 = 2^3 - 1 = 7
```

5.3 Traço de Execução para n = 2

Para compreender melhor, vamos fazer o traço manual para n = 2:

```
resolverHanoi(2, 'A', 'C', 'B')
│
├─ n=2, não é caso base
├─ PASSO 1: resolverHanoi(1, 'A', 'B', 'C')  // Mover 1 disco de A para B
│  │
│  └─ n=1, CASO BASE!
│     └─ Imprimir: "Mover disco 1 de A para B"
│
├─ PASSO 2: Imprimir: "Mover disco 2 de A para C"
│
└─ PASSO 3: resolverHanoi(1, 'B', 'C', 'A')  // Mover 1 disco de B para C
   │
   └─ n=1, CASO BASE!
      └─ Imprimir: "Mover disco 1 de B para C"

Resultado:
1. Mover disco 1 de A para B
2. Mover disco 2 de A para C
3. Mover disco 1 de B para C
```

✏️ ATIVIDADE PRÁTICA 5.1 – Execute e Analise

1. Digite o código completo da Torre de Hanói no seu IDE
2. Execute para n = 3 e verifique se a saída corresponde ao esperado
3. Execute para n = 4 e conte os movimentos. Confere com a fórmula 2^n - 1?
4. Faça o traço manual completo para n = 3 (similar ao exemplo n=2 acima)

5.4 Análise de Complexidade

Relação de recorrência:

$T(n) = 2 \cdot T(n-1) + O(1)$

$T(1) = O(1)$

Cada chamada com n discos faz:

- 2 chamadas recursivas com n-1 discos
- 1 movimento direto (O(1))

Expansão:

$$
\begin{aligned}
T(n) &= 2 \cdot T(n-1) + c \\
&= 2 \cdot [2 \cdot T(n-2) + c] + c \\
&= 4 \cdot T(n-2) + 2c + c \\
&= 4 \cdot [2 \cdot T(n-3) + c] + 3c \\
&= 8 \cdot T(n-3) + 4c + 3c \\
&= \ldots \\
&= 2^{(n-1)} \cdot T(1) + c \cdot (2^{(n-1)} + 2^{(n-2)} + \ldots + 2 + 1) \\
&= 2^{(n-1)} \cdot c + c \cdot (2^n - 1) \\
&= O(2^n)
\end{aligned}
$$

Número de movimentos:

$M(n) = 2 \cdot M(n-1) + 1$
$M(1) = 1$

Solucionando:
$M(n) = 2^n - 1$

Complexidade espacial:

A profundidade máxima da recursão é n (pilha de chamadas), então a complexidade espacial é **O(n)**.

5.5 Implicações Práticas

A Torre de Hanói ilustra perfeitamente um algoritmo com complexidade **exponencial O(2^n)**:

n (discos)	Movimentos (2^n - 1)	Tempo (1 mov/seg)
3	7	7 segundos
10	1.023	17 minutos
20	1.048.575	12 dias

30	1.073.741.823	34 anos
64	1,84 × 10^19	585 bilhões de anos

Lenda: A lenda original afirma que monges em um templo movem 64 discos de ouro. Quando terminarem, o mundo acabará. Com 1 movimento por segundo, levaria 585 bilhões de anos!

5.6 Aplicações e Conexões

A Torre de Hanói não é apenas um quebra-cabeça matemático. Tem aplicações práticas:

1. Backup e migração de dados: Estratégias de cópia incremental usando 3 mídias (similar aos 3 pinos).

2. Algoritmos de pilha: Demonstra o poder da estrutura de pilha (LIFO) em problemas recursivos.

3. Análise de algoritmos: Exemplo canônico de complexidade exponencial.

4. Testes de QI e entrevistas: Problema comum em entrevistas técnicas para avaliar pensamento recursivo.

> ✏️ **ATIVIDADE PRÁTICA 5.2 – Variação do Problema**
>
> Modifique o código para contar **quantas vezes cada disco é movido** individualmente. Crie um array `int[] movimentosPorDisco = new int[n+1]` e incremente o contador apropriado em cada movimento. O que você observa sobre a frequência de movimento de cada disco?
>
> Dica esperada: Discos menores movem mais que discos maiores. Especificamente, o disco k é movido 2^(n-k) vezes.

✅ **Checkpoint da Trilha 5:**

- ☐ Compreendo o problema da Torre de Hanói e suas regras
- ☐ Sei implementar a solução recursiva elegante
- ☐ Entendo por que a complexidade é O(2^n) e suas implicações
- ☐ Consigo fazer o traço de execução manual

TRILHA 6 – RECURSÃO vs. ITERAÇÃO (15 min)

🎯 **Objetivo:** Desenvolver critério para escolher entre soluções recursivas e iterativas.

6.1 Quando Usar Cada Abordagem?

📖 **Leia a tabela comparativa:**

Aspecto	Recursão	Iteração
Legibilidade	✓ Mais elegante para problemas naturalmente recursivos	Mais explícita no controle de fluxo
Performance	Overhead de chamadas de função	✓ Geralmente mais rápida
Memória	Usa pilha (risco overflow)	✓ Usa heap ou variáveis locais
Quando usar	Árvores, grafos, divide-and-conquer	Loops simples, grande profundidade

6.2 Convertendo Recursão para Iteração

Exemplo 1: Fatorial

```
// RECURSIVO
public int fatorialRecursivo(int n) {
    if (n <= 1) return 1;
    return n * fatorialRecursivo(n-1);
}

// ITERATIVO
public int fatorialIterativo(int n) {
    int resultado = 1;
    for (int i = 2; i <= n; i++) {
        resultado *= i;
    }
    return resultado;
}
```

Exemplo 2: Soma até N

```
// RECURSIVO
public int somaAteNRecursivo(int n) {
    if (n == 0) return 0;
    return n + somaAteNRecursivo(n-1);
}

// ITERATIVO
public int somaAteNIterativo(int n) {
    int soma = 0;
    for (int i = 1; i <= n; i++) {
        soma += i;
    }
    return soma;
}
// Ou ainda: return n * (n + 1) / 2;  // Fórmula de Gauss - O(1)!
```

Exemplo 3: Fibonacci

```
// RECURSIVO (com memoization - O(n))
private static Map<Integer, Long> memo = new HashMap<>();
public long fibRecursivo(int n) {
    if (n <= 1) return n;
    if (memo.containsKey(n)) return memo.get(n);
    long resultado = fibRecursivo(n-1) + fibRecursivo(n-2);
    memo.put(n, resultado);
    return resultado;
}

// ITERATIVO - O(n) tempo, O(1) espaço
public long fibIterativo(int n) {
    if (n <= 1) return n;

    long anterior = 0;
    long atual = 1;
```

```
    for (int i = 2; i <= n; i++) {
        long proximo = anterior + atual;
        anterior = atual;
        atual = proximo;
    }

    return atual;
}
```

Observação importante: A versão iterativa de Fibonacci usa **O(1) espaço** (apenas 3 variáveis), enquanto a recursiva com memoization usa **O(n) espaço** (HashMap + pilha).

6.3 Quando a Recursão é Essencial

Alguns problemas são **inerentemente recursivos** e convertê-los para iteração é muito mais complexo:

```
1. Percorrer árvores (próximas semanas):

// Percurso in-order em árvore binária
public void inOrder(Node raiz) {
    if (raiz == null) return;
    inOrder(raiz.esquerda);
    System.out.println(raiz.valor);
    inOrder(raiz.direita);
}
```

2. Torre de Hanói:

A solução iterativa existe, mas é significativamente mais complexa e menos intuitiva que a recursiva.

3. Quicksort e Merge Sort:

Algoritmos de ordenação divide-and-conquer são naturalmente recursivos. A versão iterativa requer uso explícito de pilha.

Conclusão prática: Use recursão quando:

- O problema tem estrutura naturalmente recursiva (árvores, grafos)
- A solução recursiva é significativamente mais simples
- A profundidade máxima é controlada (não risco de stackoverflow)

Use iteração quando:

- Performance é crítica
- A profundidade pode ser muito grande
- A conversão para iterativo é simples e direta

✏️ **ATIVIDADE PRÁTICA 4.1 – Converta para Iterativo**

Converta a função fatorial recursiva para versão iterativa usando um loop for:

```
public int fatorialIterativo(int n) {    // Seu código aqui}
```

Dica: Use uma variável acumuladora inicializada em 1 e multiplique por cada i de 2 até n.

Tempo: 10 minutos

✅ **Checkpoint da Trilha 6:**

- Compreendo as vantagens e desvantagens de recursão vs iteração
- Sei converter funções recursivas simples para iterativas
- Reconheço quando recursão é a escolha natural

EXERCÍCIOS PRÁTICOS

Entregar os códigos logo abaixo de cada exercício

Exercício 1: Implemente uma função recursiva que calcule a potência x^n. Analise sua complexidade.

```
//cole sue código dentro da área cinza usando CTRL+SHIFT+V
```

Exercício 2: Escreva uma função recursiva que inverta uma string.

```
//cole sue código dentro da área cinza usando CTRL+SHIFT+V
```

Exercício 3: Implemente a busca binária recursiva em um array ordenado. Qual a relação de recorrência? Aplique o Teorema Mestre.

```
//cole sue código dentro da área cinza usando CTRL+SHIFT+V
```

Exercício 4: Determine a complexidade da seguinte recorrência: $T(n) = 3 \cdot T(n/4) + n^2$

```
//cole sue código dentro da área cinza usando CTRL+SHIFT+V
```

Exercício 5: Compare a performance das implementações recursiva e iterativa de Fibonacci para n = 40. O que você observa?

```
//cole sue código dentro da área cinza usando CTRL+SHIFT+V
```

Exercício 6: Implemente a Torre de Hanói recursivamente e determine sua complexidade. Calcule:

- Quantos movimentos são necessários para n = 5?
- Quanto tempo levaria para n = 30 discos se cada movimento leva 1 segundo?
- Modifique o código para imprimir o estado das 3 torres após cada movimento.

```
//cole sue código dentro da área cinza usando CTRL + SHIFT + V
```

Exercício 7: (Desafio) Implemente a função de Ackermann:

```
A(m, n) = n + 1                    se m = 0
A(m, n) = A(m-1, 1)                se m > 0 e n = 0
A(m, n) = A(m-1, A(m, n-1))        se m > 0 e n > 0
```

Teste para A(3, 4) e observe a explosão de chamadas. Esta função cresce MUITO mais rápido que qualquer função exponencial!

```
//cole sue código dentro da área cinza usando CTRL + SHIFT + V
```

CHECKLIST DE CONCLUSÃO DO ESTUDO

Antes de encerrar seu estudo autônomo, verifique se você consegue:

FUNDAMENTOS

- ☐ Explicar o que é recursão com suas próprias palavras
- ☐ Identificar caso base, caso recursivo e progresso em qualquer função recursiva
- ☐ Fazer traço de execução manual de uma função recursiva simples

PILHA E ERROS

- ☐ Explicar como a pilha de chamadas funciona
- ☐ Identificar código que causaria StackOverflowError
- ☐ Escrever casos base robustos que previnem erros

OTIMIZAÇÃO

- ☐ Implementar Fibonacci com memoization usando HashMap
- ☐ Explicar por que memoization reduz complexidade de $O(2^n)$ para $O(n)$

RECURSÃO vs ITERAÇÃO

- ☐ Converter função recursiva simples para iterativa
- ☐ Decidir quando usar recursão vs. iteração baseado no problema